

Generalization and Regularization

Diagnose the gap · change one lever · measure again

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Four lectures made training loss go down — that was the easy part

Keep from L3–L6

backprop measures g_t , AdamW turns it into reliable updates, initialization and normalization keep both alive through depth

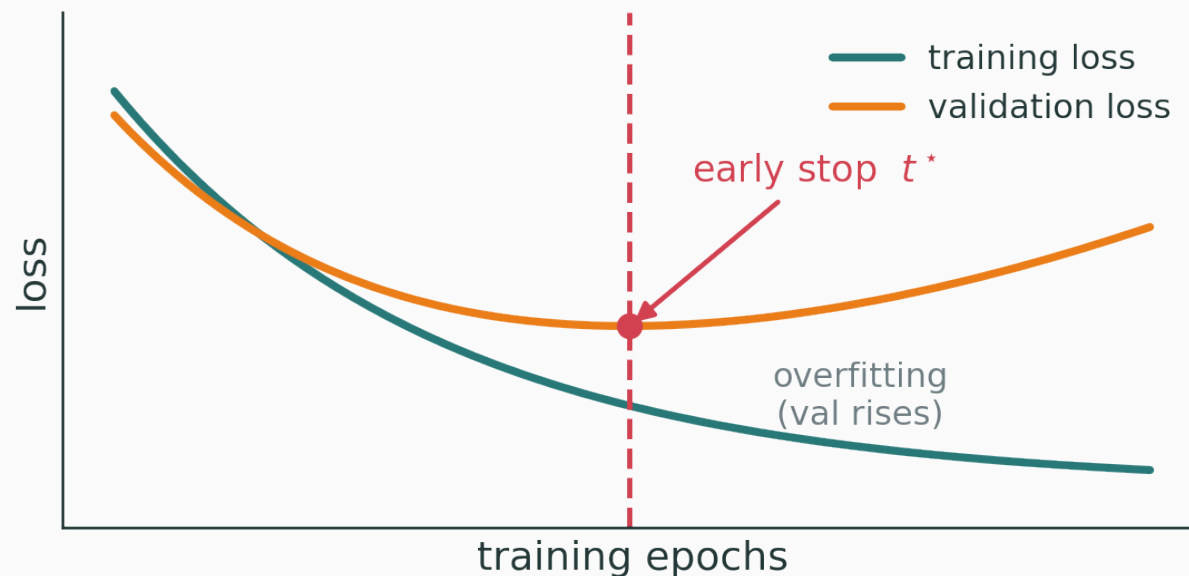
$$\min_{\theta} \mathcal{L}_{\text{train}}(\theta) \neq \min_{\theta} \mathcal{L}_{\text{new data}}(\theta)$$

Today's problem

the number those tools minimize is computed on data the model has **already seen**

Today's question: when driving $\mathcal{L}_{\text{train}}$ down stops helping, **what else can training express — and where does it enter the pipeline?**

Start here: the two curves disagree



$\mathcal{L}_{\text{train}} \downarrow$ throughout, while $\mathcal{L}_{\text{val}} \downarrow$ then $\uparrow$: the model keeps fitting – but what it learns stops transferring.

Three splits, three different jobs

split	job	allowed decisions
training	fit parameters θ	gradients; optimizer updates
validation	choose the recipe	λ , stopping time, p , augmentation, ε
test	estimate final generalization	none — spend once after the recipe is frozen

Using test accuracy to choose even one regularization knob turns the test set into another validation set. Our toy spine therefore seals a separate dense test draw until the finale.

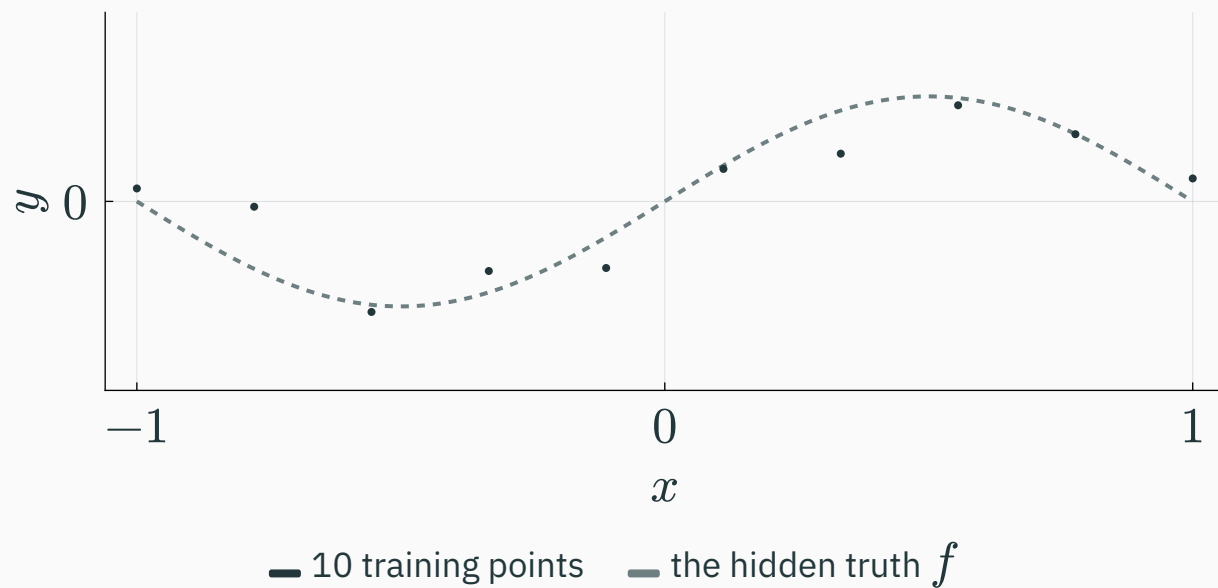
Outline

1. **Diagnose** — one tiny dataset, a perfect fit, and the gap that exposes it
2. **Weight decay** — charge the objective for coefficient magnitude; AdamW
3. **Early stopping** — treat training time itself as a capacity knob
4. **Augmentation + Mixup/CutMix** — encode task-valid invariances
5. **Dropout** — train overlapping subnetworks; prepare MC Dropout
6. **Label smoothing** — soften targets and temper confidence

Ten measurements and a model family you already know

We measure a smooth signal at ten inputs; every reading carries noise:

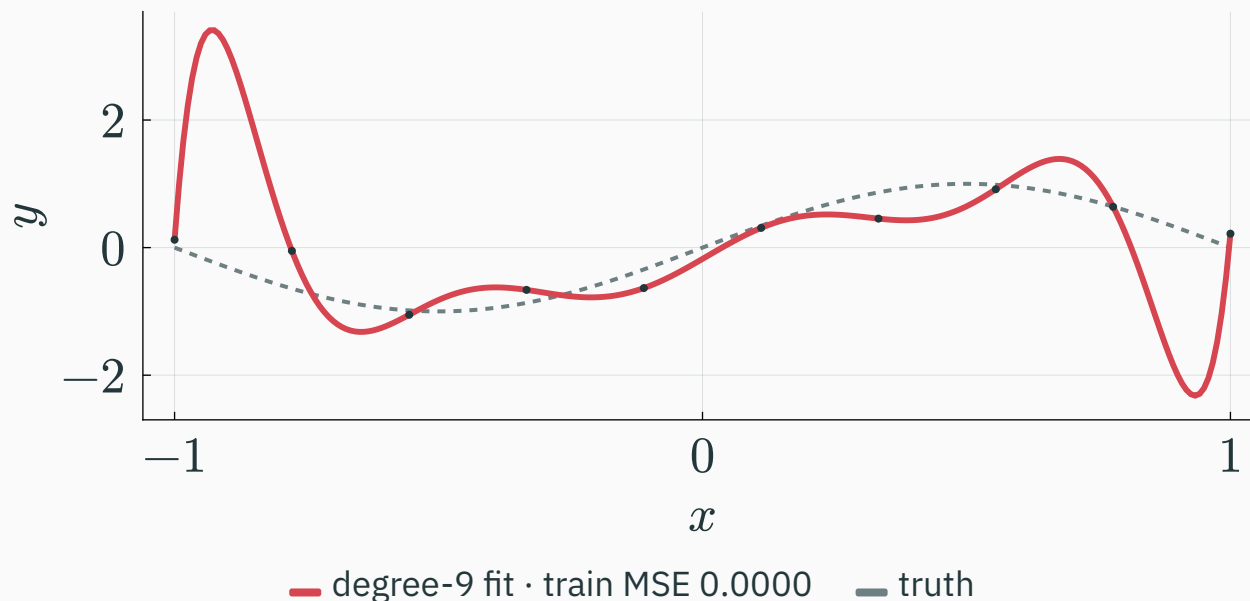
$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, 0.25^2).$$



From ES 335: expand features as $\varphi(x) = (1, x, x^2, \dots, x^9)$ and fit $\hat{y} = \mathbf{w}^\top \varphi(x)$ — a linear model with 10 weights, flexible enough to bend anywhere.

The training loss reaches zero — exactly

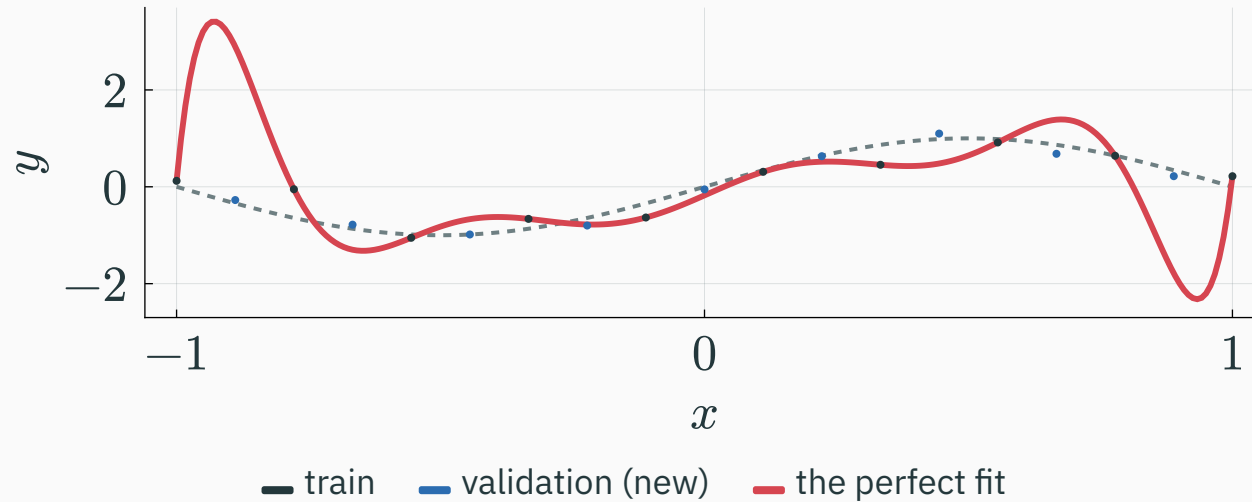
Ten equations, ten weights: the least-squares fit passes through **every** point.



A training loss of 0.0000 is a red flag, not a finish line.

Nine points in between expose the failure

Measure the same signal at the nine **midpoints** the model never saw:



degree-9 fit	train MSE	validation MSE
same ten weights	0.0000	1.70

Even the true f would score $\approx \sigma^2 = 0.0625$ here; the perfect memorizer is $27 \times$ worse — **between** the points it memorized.

Learning curves separate three diagnoses

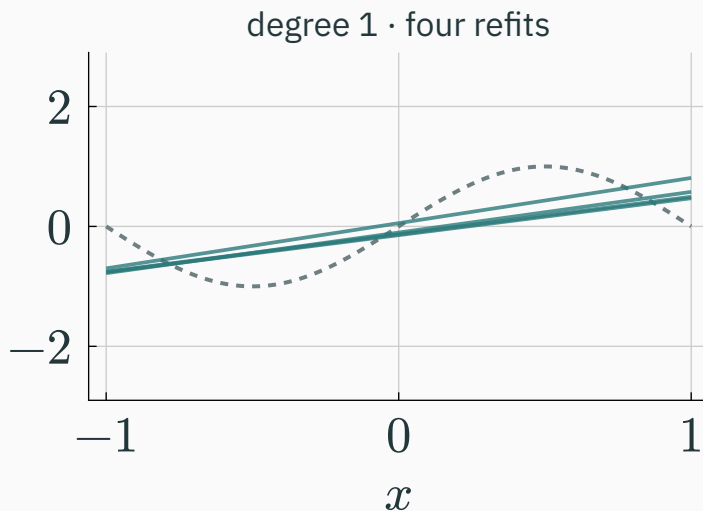
The pair $(\mathcal{L}_{\text{train}}, \mathcal{L}_{\text{val}})$ — read levels, gap, and trend:

training	validation	diagnosis	first move
high	high, similar	underfit or broken optimization	repair fit (L5–L6)
low	much higher	overfitting	regularize
low	low, similar	fit transfers to this split	stress- test, ship

A small gap alone is not success — both losses can agree because both are bad.
Our spine: train 0.00, validation 1.70 — the second row.

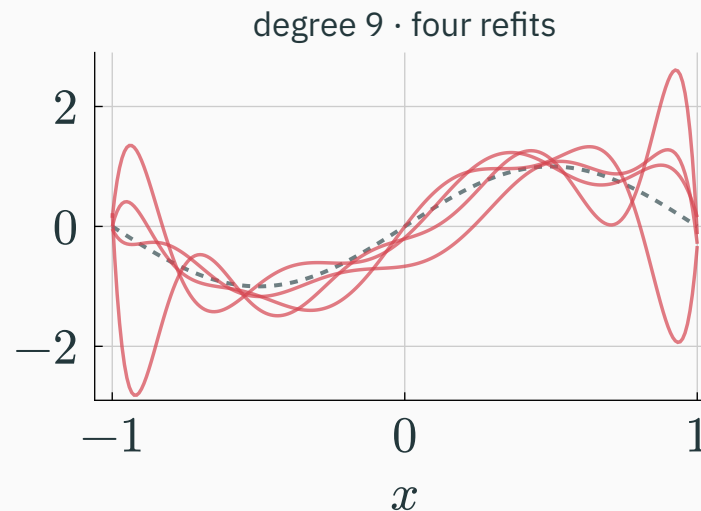
Same recipe, fresh noise: the fit is nervous

Re-draw the ten noise values four times and refit each dataset:



Rigid model

barely reacts to the noise — but cannot express the signal (**bias**)

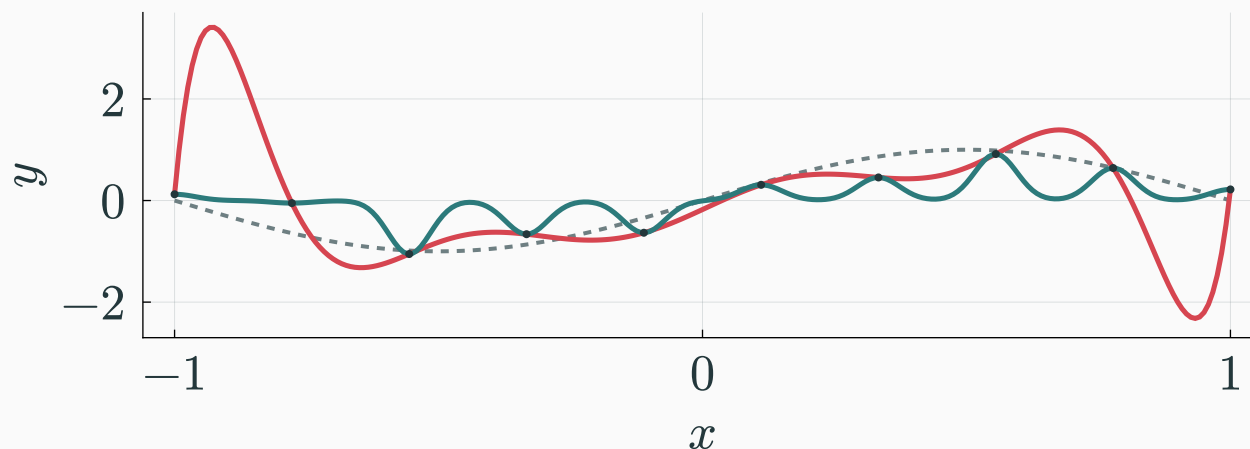


Flexible model

swings with every noise draw — the noise **is** in the fit (**variance**)

Both of these fits have zero training loss

Narrow radial bumps through the same ten points **also** score 0.0000:

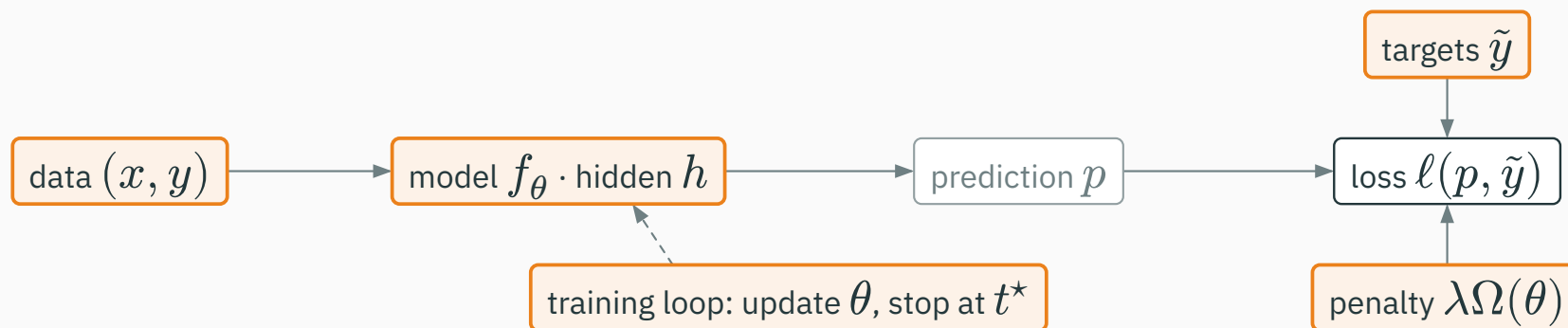


— polynomial interpolant — narrow-bump interpolant — both: train MSE = 0

Training data cannot choose between them. They differ exactly where the data is silent.

Modern deep networks live here: many settings reach zero training error — even on **random labels** (Zhang et al., 2017). Which rule you get is decided by everything else about training.

Regularization is a policy for the silent regions



Each method today is one **preference for simpler behaviour**, injected at one site, with one knob — and **validation** sets every knob.

— training / fit — preference + knob — validation — memorization — kept model

**Weight decay: charge the
objective for magnitude**

Add the preference directly to the objective

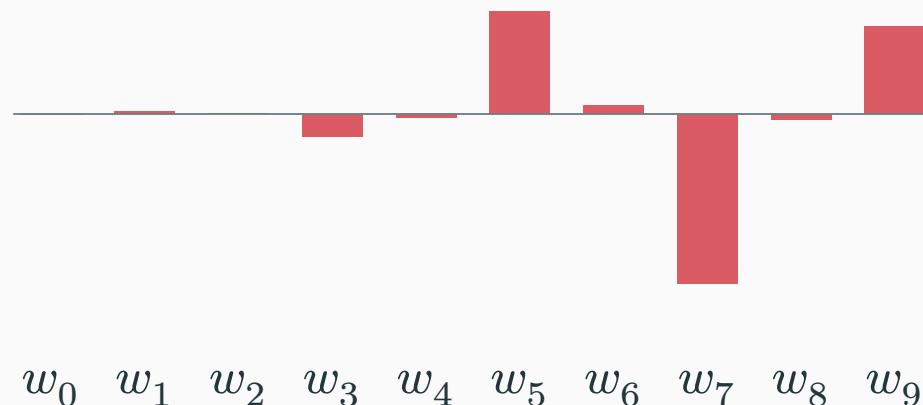


$$\mathcal{L}_{\text{data}(w)} = \frac{1}{2n} \|Xw - y\|_2^2, \quad \mathcal{J}(w) = \underbrace{\mathcal{L}_{\text{data}(w)}}_{\text{fit}} + \underbrace{\frac{\lambda_{\text{wd}}}{2} \|w\|_2^2}_{\text{shrink}}$$

Convention: the $\frac{1}{2}$ factors make $\nabla \mathcal{J} = \frac{X^\top(Xw - y)}{n} + \lambda_{\text{wd}}w$. Lecture 1 derived the Gaussian-prior view; today we watch what the same term does to a fit and an optimizer. Revisit: [interactive-articles/map-regularization](#)

What the perfect fit paid in coefficients

The zero-training-loss polynomial needed enormous, cancelling weights:



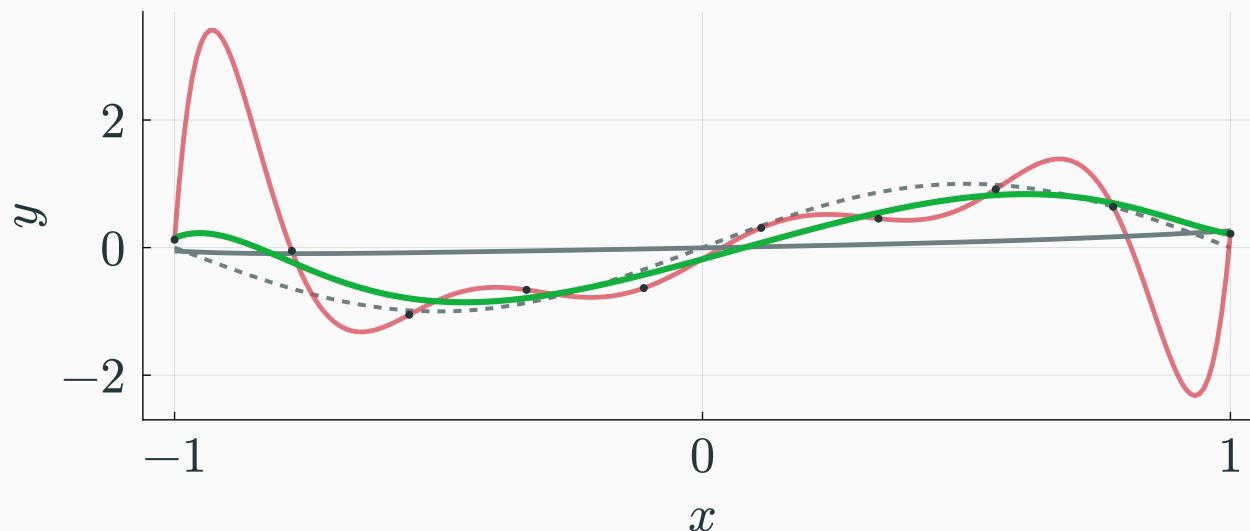
weights of the degree-9 interpolant · largest 362 in magnitude · $\|w\|_2 = 465$

Wild wiggles require huge coefficients. Penalizing magnitude is a direct tax on wiggling.

Caveat: parameter norm is not function smoothness (ReLU layers can rescale freely) — it is a useful preference, and validation decides if it helps.

One knob sweeps the family from wiggle to flat

Same ten points, same ten features — only λ changes:



— $\lambda = 0 \cdot \|w\| = 465$ — $\lambda = 10^{-3} \cdot \|w\| = 3.9$ — $\lambda = 1 \cdot \|w\| = 0.2$

λ is an exchange rate between the two forces: 0 returns the memorizer, too much buys back the rigid model.

Checkpoint: which λ do we keep?

The sweep on the spine (all values computed live):

λ	train MSE	val MSE	$\ w\ _2$
10^{-6}	0.014	0.151	35.1
10^{-4}	0.018	0.077	5.6
10^{-3}	0.023	0.065	3.9
10^{-2}	0.062	0.111	1.8
10^0	0.286	0.405	0.2

Which column is allowed to choose λ ?

A. train MSE — it is the objective we optimize

B. validation MSE

C. $\|w\|_2$ — smaller is safer

D. none; average all three columns

Answer: validation chooses; this data loss votes $\lambda = 0$

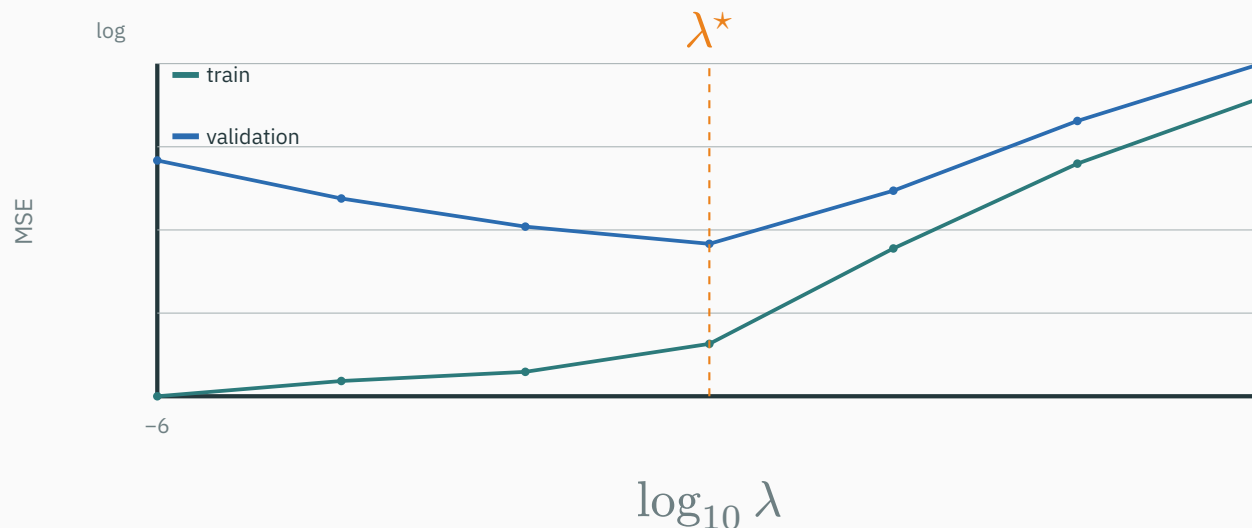
A ANSWER

Correct: B — validation MSE picks $\lambda^* = 10^{-3}$

Why: in this ridge sweep the **unpenalized** training MSE is minimized at $\lambda = 0$; the weight norm is a mechanism check, not a target.

This selection loop — sweep the knob, read **validation — repeats for **every** regularizer today.**

The U-curve is the pattern to look for — not a law



Left of λ^* : variance dominates (memorization). Right: bias dominates (back to rigid). Other regularization knobs often show a related sweet spot, but real validation curves can be noisy, flat, or multi-modal — sweep and measure.

Under SGD, the penalty becomes multiplicative shrinkage

The penalty's gradient is just $\lambda\theta_t$, so the SGD step factors:

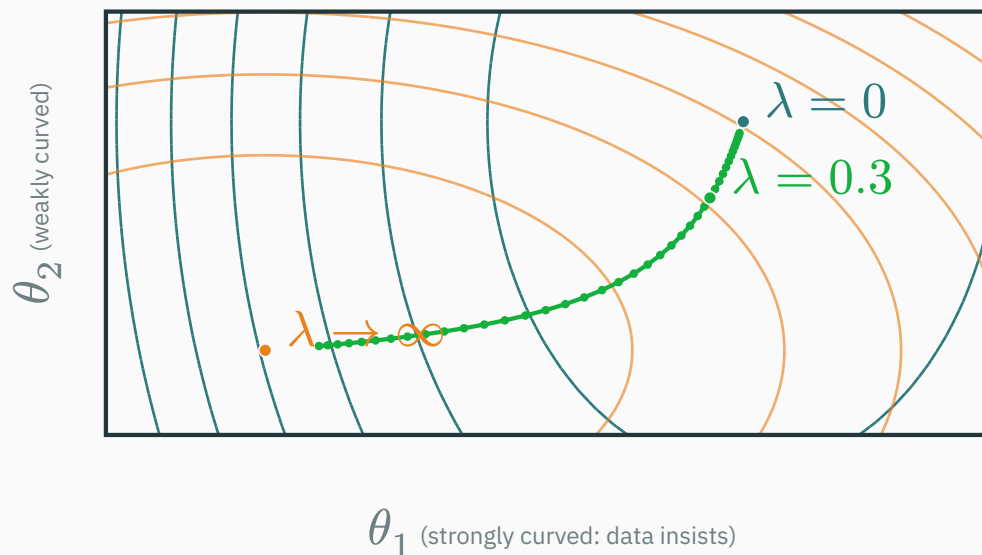
$$\theta_{t+1} = \theta_t - \eta(g_t + \lambda\theta_t) = \underbrace{(1 - \eta\lambda)\theta_t}_{\text{shrink first}} - \underbrace{\eta g_t}_{\text{then fit}}$$

One concrete step with $\theta_t = 5$, $g_t = 3$, $\eta = 0.1$, $\lambda = 0.2$:

piece	substitution	value
shrink	$(1 - 0.1 \cdot 0.2) \times 5 = 0.98 \times 5$	4.90
fit	-0.1×3	-0.30
θ_{t+1}	$4.90 - 0.30$	4.60

Each step, every weight quietly loses 2% of itself – hence **weight decay.**

Quadratic data loss (curvature 4 along θ_1 , 0.6 along θ_2) plus the penalty's circles:



The path bends: the **weakly curved** coordinate collapses first (θ_2 : 1.35 $\rightarrow$ 0.90 already at $\lambda = 0.3$), while the direction the data insists on survives longest. Decay suppresses what the data barely constrains.

Checkpoint: send the same penalty through Adam

QUESTION

Recall Lecture 5. Two weights $\theta = (1, 1)$, but Adam's scale memory differs: $\hat{v} = (16, 0.16)$. Take a one-step snapshot with $\eta = 0.01$, $\lambda = 0.1$, and a frozen diagonal preconditioner $\hat{v}$.

Ignoring first-moment dynamics and ε to isolate the effect: if $\lambda\theta$ rides through the adaptive update, its contribution is $-\eta\lambda\theta/\sqrt{\hat{v}}$.

How do the two shrinkage contributions compare?

A. both shrink by 0.001 — the penalty treats equals equally

B. the high- $\hat{v}$ weight shrinks $10 \times$ more

C. the low- $\hat{v}$ weight shrinks $10 \times$ more

D. neither shrinks; Adam cancels the penalty exactly

Answer: the preconditioner distorts the intended preference

A ANSWER

L_2 inside Adam

$$-0.01(0.1)/\sqrt{16} = -0.00025$$

$$-0.01(0.1)/\sqrt{0.16} = -0.0025$$

Decoupled (AdamW)

$$-\eta\lambda\theta_1 = -0.001$$

$$-\eta\lambda\theta_2 = -0.001$$

Correct: C — the low- $\hat{v}$ weight shrinks $10 \times$ more

Why: dividing by $\sqrt{\hat{v}}$ rescales the penalty per-coordinate — you asked for proportional shrinkage and got something else.

AdamW: decay the weights, precondition only the data gradient

D DERIVATION

$$\theta_{t+1} = \underbrace{(1 - \eta\lambda)\theta_t}_{\text{decay, undistorted}} - \underbrace{\eta \text{AdamUpdate}_t(g)}_{\text{adaptive data step}}$$

plain SGD	L_2 penalty and weight decay are the same algebra
adaptive (Adam)	they differ ; decoupling restores the intended preference (AdamW)

Practice notes: `weight_decay` in `torch.optim.AdamW` is this decoupled λ ; biases and normalization gains are commonly excluded from decay — a recipe choice to log, not a theorem. Loshchilov & Hutter (2019) · PyTorch AdamW API.

PyTorch: make the decay policy visible in parameter groups

```
decay, no_decay = [], []
for name, p in model.named_parameters():
    if not p.requires_grad:
        continue
    # Common recipe: do not decay bias or 1-D scale/shift parameters.
    (no_decay if p.ndim == 1 or name.endswith(".bias") else decay).append(p)

optimizer = torch.optim.AdamW([
    {"params": decay, "weight_decay": 1e-4},
    {"params": no_decay, "weight_decay": 0.0},
], lr=3e-4)
```

The exclusion is a **recipe**, not a theorem. Log the groups, learning-rate schedule, and λ_{wd} together; AdamW's effective shrink per step is $\eta_t \lambda_{\text{wd}}$.

penalty	gradient force	pressure	role in DL
$\lambda \ \theta\ _2^2 / 2$	$\lambda \theta$ — proportional	shrink everything a little	default
$\lambda \ \theta\ _1$	$\lambda \operatorname{sign}(\theta)$ — subgradient	promote sparse solutions	specialized

From Lecture 1: L_2 is a Gaussian prior, L_1 a Laplace prior. Ordinary SGD need not land exactly on zero; proximal/thresholding methods or explicit pruning create exact zeros. In standard dense networks, L_1 is secondary.

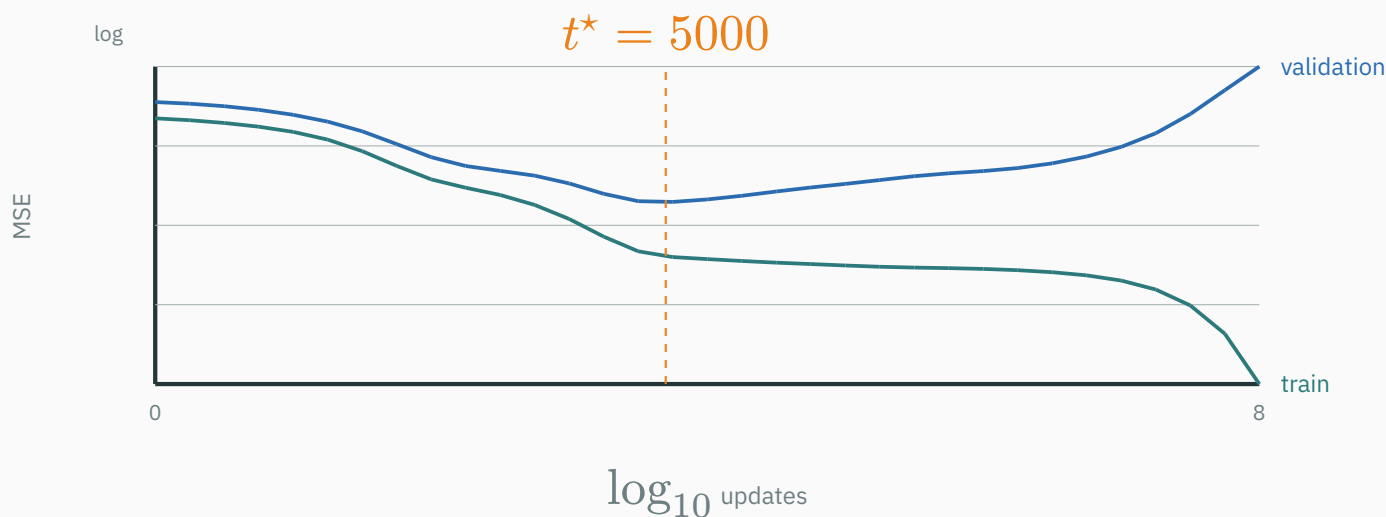
Interactive: drag λ yourself — [interactive-articles/weight-decay](https://interactive-articles.github.io/weight-decay/)

**Early stopping: training time is
a capacity knob**

Train the spine by gradient descent and watch both losses



Same model, **no penalty** — just gradient descent from $w = 0$, both losses logged:



Train falls forever. **Validation** falls, turns at t^* , and climbs — the U-curve again, with **time** on the axis.

The rule: keep the checkpoint validation preferred

$$t^* = \arg \min_t \mathcal{L}_{\text{val}}(w_t)$$

checkpoint t	train MSE	val MSE	verdict
100	0.097	0.153	discard
1000	0.043	0.089	discard
5000	0.020	0.061	restore this one
30000	0.018	0.075	discard
1000000	0.016	0.115	discard

```
from copy import deepcopy
best = patience_ref = float("inf")
wait, best_state = 0, None

for epoch in range(max_epochs):
    train_one_epoch(model, train_loader, optimizer)
    val_loss = evaluate(model, val_loader)

    if val_loss < best:                # strict argmin checkpoint
        best = val_loss
        best_state = deepcopy(model.state_dict())

    if val_loss < patience_ref - min_delta:
        patience_ref, wait = val_loss, 0 # meaningful progress
    else:
        wait += 1
        if wait >= patience: break

model.load_state_dict(best_state) # restore; do not keep the final epoch
```

Checkpointing tracks the strict arg min; min_delta only controls patience. To resume training later, also save optimizer, scheduler, scaler, epoch, and RNG states.

Why validation turns: directions arrive in order of strength

For quadratic loss, gradient descent from $w_0 = 0$ has an exact per-direction solution. Along eigendirection v_j of the data matrix (strength σ_j^2):

$$v_j^\top w_t = \left(1 - (1 - \eta\sigma_j^2)^t\right) \cdot v_j^\top w_\infty$$

Strong directions (σ_j^2 large)

factor $\rightarrow 1$ within a few updates — the **shared, simple** structure

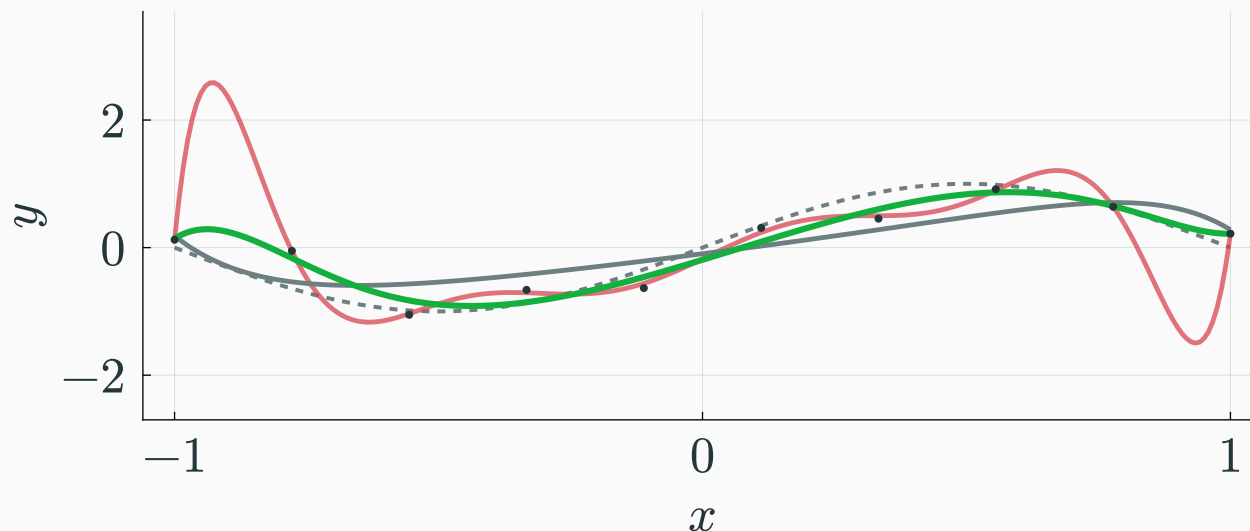
Weak directions (σ_j^2 tiny)

need $t \approx 1/(\eta\sigma_j^2)$ updates — where the **noise** hides

Our spine's ten strengths, computed: $\sigma^2 = 1.56, 1.18, 0.36, \dots, 7 \times 10^{-8}$ — a **ten-million-fold** spread. So there is a long window where signal has arrived and noise has not. Exact for linear least squares; “networks learn simple patterns first” is the empirical deep-learning analogue.

Watch the stopping time act as a capacity knob

Three checkpoints of the **same run** — no penalty anywhere:

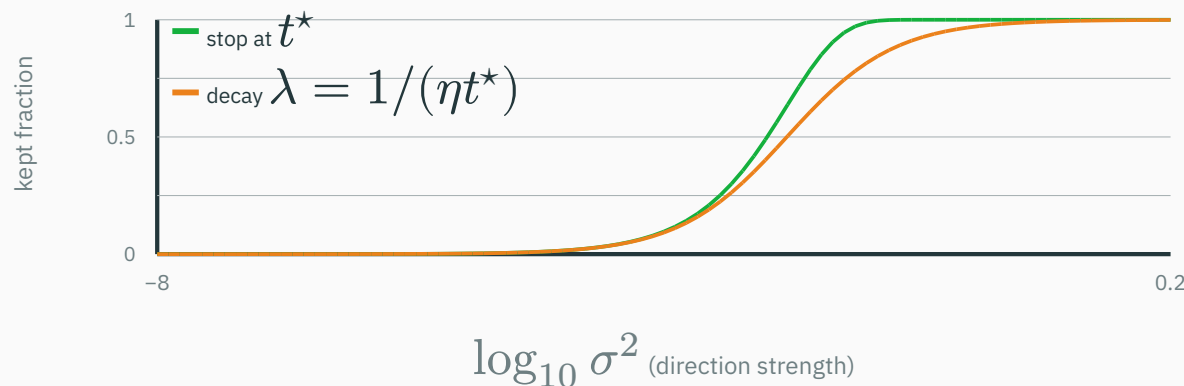


— $t = 100$ · still arriving — $t^* = 5000$ · kept — $t = 10^8$ · the noise arrived — slide 4's memorizer, reborn

Stop early → the weak, noise-carrying directions simply never arrive.

Per direction, the two methods apply strikingly similar suppression factors:

$$\underbrace{1 - (1 - \eta\sigma^2)^t}_{\text{early stopping}} \quad \text{vs.} \quad \underbrace{\frac{\sigma^2}{\sigma^2 + \lambda}}_{\text{weight decay}}$$



On our spine the match is uncanny: validation independently chose $\lambda^* = 10^{-3}$ and $t^* = 5000$ — and $1/(\eta t^*) = 0.0012$. Two knobs, one preference.

INTERACTIVE

↗ colab.research.google.com/github/nipunbatra/dl-teaching/blob/master/notebooks/L07/01_overfitting_and_early_stopping.ipynb

A NumPy lab that makes the mechanism inspectable: a flexible model overfits a small dataset while you log both curves and restore the best checkpoint.

Before executing each cell, write down:

- which epoch range the validation minimum will fall in;
- what **patience** value would have caught it;
- what happens to the restored-checkpoint test error if you double the noise.

**Augmentation: teach
invariances through the data**

One cat, six views — which changes should the classifier ignore?

six training views · one label: cat



CIFAR-10 · 32×32 source image · every transform must preserve the task label

Augmentation is not decoration: each view is a claim that the answer stays **cat.**

real CIFAR-10 training image · transforms regenerated locally · inspectability matters more than photorealism at

32×32

The strongest practical regularizer edits the data distribution



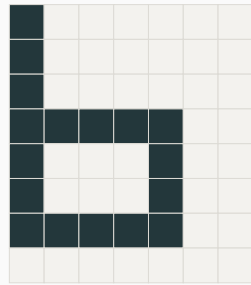
Declare a distribution $\mathcal{T}$ of transformations. For a label-preserving classification transform, $y' = y$; for structured targets, transform the pair:

$$(x', y') = T(x, y), \quad \min_{\theta} \mathbb{E}_{(x, y)} \mathbb{E}_{T \sim \mathcal{T}} [\ell(f_{\theta(x')}, y')]$$

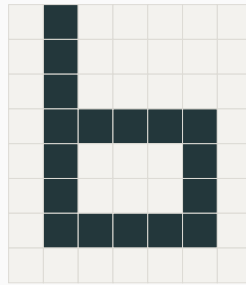
Every epoch samples a new training view: Monte Carlo over your declared invariances. The stored dataset is finite; the **training distribution** no longer is. In vision this is often the highest-leverage regularizer.

A transform is a claim about the label

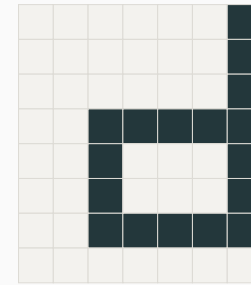
Pixel letters, class set $\{a, b, c, d, e\}$ — watch what each transform claims:



original — label “b”



shifted — still “b”



flipped — now reads “d”

Horizontal flip is a **valid invariance for flowers — and a **label change** for letters.**

Which policy states an invariance the letter labels actually possess?

A. horizontal flip — objects don't care about left and right

B. rotate by 180° — orientation is superficial

C. shift by one or two pixels, keeping the glyph in frame

D. any transform that makes training loss go up

| Commit, and name the counterexample for each rejected option.

Correct: C — small shifts keep the glyph and the label

Why: flip turns $b \leftrightarrow d$; a 180° rotation turns 6 into 9; “harder training” is not a label-preservation argument.

transform	claims the task ignores...	breaks when...
random crop	position and scale (a little)	the crop removes the object
horizontal flip	left–right orientation	text, digits, anatomy (the heart has a side)
colour jitter	illumination and tint	colour is the diagnosis

detection reminder: geometric transforms must move boxes and masks too — augment the **supervision**

The label contract changes with the task

Classification

crop or flip the image; the class index usually stays fixed

Detection / segmentation

move boxes, masks, and keypoints with the pixels

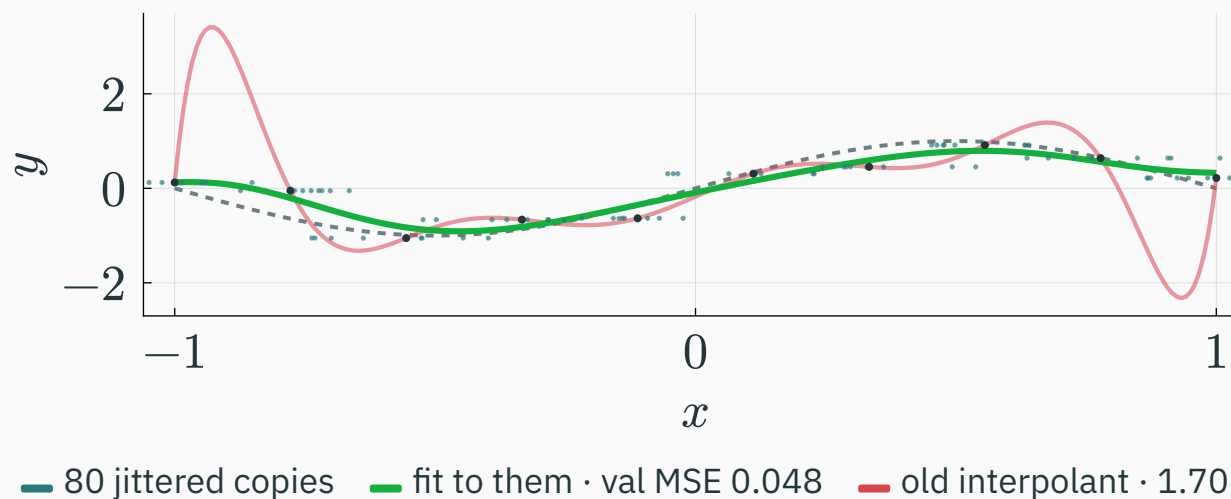
Audio / text

mask time–frequency bands or edit tokens only when meaning survives

Applying a geometric transform to x but not to a spatial target y creates **wrong labels**, not useful noise. Prefer transform APIs that accept the whole sample.

Input jitter imposes local smoothness through the data

Impose local smoothness: reuse each measured y at 8 nearby inputs $x + \varepsilon$. This is a deliberate approximation — not an exact invariance of $\sin(\pi x)$:



The fit becomes locally smoother, at the cost of some bias; validation judges the trade. Input noise is closely related to a smoothness penalty (Bishop, 1995).

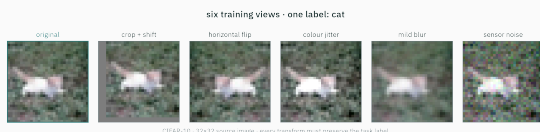
```
from torchvision.transforms import v2

train_tf = v2.Compose([
    v2.ToImage(),
    v2.RandomCrop(32, padding=4, padding_mode="reflect"),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean, std),
])

eval_tf = v2.Compose([
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean, std),
])
```

Train sees fresh random views; validation/test use a deterministic pipeline.
Compute mean, std from the training split only. torchvision v2 reference

Before training, audit an augmented batch



For eight random views, ask:

- Is the object still present?
- Would a domain expert keep the label?
- Are boxes or masks still aligned?
- Is the strength plausible at deployment?
- Is normalization applied **after** pixel transforms?

```
views = torch.stack([train_tf(raw_image) for _ in range(8)])
grid = torchvision.utils.make_grid(denormalize(views), nrow=4)
show(grid)                                     # make this a pre-flight check
```

Mixup: train on blends of examples and labels

D DERIVATION

Interpolate **both** the inputs and the one-hot targets, with $\lambda \sim \text{Beta}(\alpha, \alpha)$:

$$\tilde{x} = \lambda_{\text{mix}} x_i + (1 - \lambda_{\text{mix}}) x_j, \quad \tilde{y} = \lambda_{\text{mix}} y_i + (1 - \lambda_{\text{mix}}) y_j$$

mix the supervision whenever you mix the image

cat



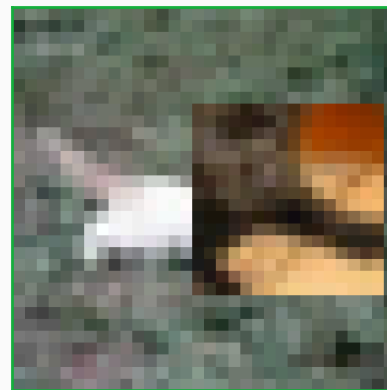
dog



MixUp
0.70 cat + 0.30 dog



CutMix
0.75 cat + 0.25 dog



Between examples — exactly where data was silent — the model is now told to behave **linearly**.

The mixed target is exactly the mixing weight

For $\lambda_{\text{mix}} = 0.70$ mixing a cat with a dog:

$$\tilde{y} = 0.70 y_{\text{cat}} + 0.30 y_{\text{dog}}$$

The cross-entropy against a soft target splits by linearity:

$$\ell(\tilde{y}, p) = -0.70 \log p_{\text{cat}} - 0.30 \log p_{\text{dog}}$$

Two checkable consequences: the loss never reaches zero (even a perfect $p = \tilde{y}$ scores 0.611, the target entropy), and extreme confidence on either class is punished. MixUp regularizes targets **and** inputs.

```
from torchvision.transforms import v2
import torch.nn.functional as F

batch_aug = v2.RandomChoice([
    v2.MixUp(num_classes=K, alpha=0.2),
    v2.CutMix(num_classes=K, alpha=1.0),
])

model.train()
for images, class_ids in train_loader:
    images, soft_targets = batch_aug(images, class_ids)
    logits = model(images)
    loss = F.cross_entropy(logits, soft_targets)
    optimizer.zero_grad(set_to_none=True)
    loss.backward(); optimizer.step()
```

The transform pairs examples within a batch and returns probability targets; `cross_entropy` accepts them directly. Accuracy still uses the original class IDs on an unmodified evaluation set. official example

extension	one-line mechanism
CutMix	paste a patch of image j onto image i ; mix labels by pasted-area fraction — keeps local evidence crisp
RandAugment	sample a couple of transforms and one global magnitude from a small policy space — augmentation search without the search cost

Both inherit augmentation's contract: the arithmetic is automatic, but the transformation space and its strength remain validation-selected claims about the task.

**Dropout: train with randomly
deleted features**

A feature that only works in committee is fragile



Co-adaptation

with all units always present, a hidden feature can lean on its neighbours — a brittle committee that memorizes together

The intervention

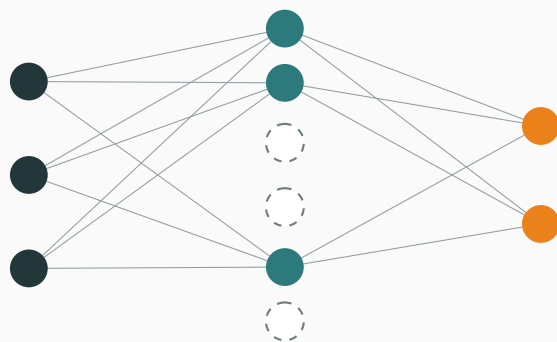
delete a random subset of features at **every** training step, so each must carry evidence on its own

If any teammate might vanish, every feature must be individually useful.

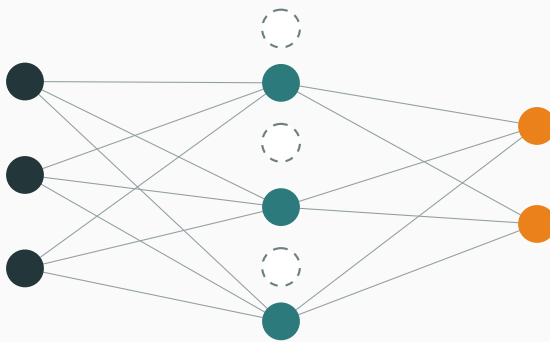
The mechanism is one random mask

Sample a fresh Bernoulli mask each pass; keep units with probability q :

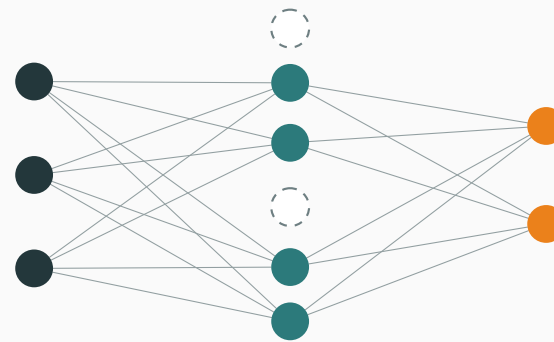
$$m_i \sim \text{Bernoulli}(q), \quad \tilde{h} = \frac{m \odot h}{q}$$



step 1 · mask 110010



step 2 · mask 010101



step 3 · mask 011011

three consecutive training steps, masks drawn at $q = 0.5$ – a **different subnetwork** trains each step, all sharing one weight set

Divide by q so the expected signal is unchanged

Why the $1/q$? Take the expectation over masks:

$$\mathbb{E}[\tilde{h}_i] = \frac{\mathbb{E}[m_i] \cdot h_i}{q} = \frac{q \cdot h_i}{q} = h_i \quad \checkmark$$

One concrete pass with $\mathbf{h} = (2, 4, 6, 8)$, mask $(1, 0, 1, 0)$, $q = 0.5$:

$$\tilde{\mathbf{h}} = \frac{1 \cdot 2, 0 \cdot 4, 1 \cdot 6, 0 \cdot 8}{0.5} = (4, 0, 12, 0)$$

Corrected **in expectation** only — the injected variance $((1 - q)/q) h_i^2$ remains:
deliberate multiplicative noise.

```
def inverted_dropout(h, p_drop=0.3, training=True):
    if not training or p_drop == 0:
        return h
    if not 0 <= p_drop < 1:
        raise ValueError("p_drop must be in [0, 1)")

    q_keep = 1.0 - p_drop
    keep = torch.rand_like(h) < q_keep
    return h * keep / q_keep

h = torch.tensor([2., 4., 6., 8.])
draws = torch.stack([inverted_dropout(h) for _ in range(20_000)])
draws.mean(0)  # approximately tensor([2, 4, 6, 8])
```

The mask has no gradients and no learnable parameters. The same learned weights receive gradients through many sampled subnetworks.

Put dropout where representation redundancy exists

```
class Classifier(nn.Module):
    def __init__(self, d, K):
        super().__init__()
        self.head = nn.Sequential(
            nn.Linear(d, 256),
            nn.ReLU(),
            nn.Dropout(p=0.3),    # p_drop=.3; q_keep=.7
            nn.Linear(256, K),
        )

    def forward(self, features):
        return self.head(features)
```

Start modestly: $p \approx 0.1$ – 0.3 in modern blocks; larger values are common in wide dense heads.

Do not drop the final logits. With functional `F.dropout`, pass `training=self.training` explicitly.

One network, 2^d overlapping committees

Sampling view

d droppable units define
up to 2^d subnetworks;
each step trains one

Sharing view

all subnetworks reuse
one parameter set — no
extra memory

Test-time view

running the full network
 $\approx$ averaging the
committee

The averaging claim is exact for a single linear layer and an **approximation** for deep nonlinear networks — a useful intuition, not an identity (Srivastava et al., 2014).

Checkpoint: what does `Dropout(p = 0.5)` do in `eval()` mode?

QUESTION

PyTorch's p is the **drop** probability, so $q = 1 - p$.

```
h = torch.tensor([2., 4., 6., 8.])
drop = nn.Dropout(p=0.5)
drop.train(); drop(h)    # one draw gave: tensor([ 4.,  0., 12.,  0.])
drop.eval(); drop(h)     # = ?
```

What does the `eval()` line return?

A. (2, 4, 6, 8) — the identity

B. (1, 2, 3, 4) — scaled by q

C. (4, 8, 12, 16) — scaled by $1/q$

D. a fresh random mask, like training

Correct: A — the identity — dropout is disabled at test time

Why: inverted dropout already paid the $1/q$ correction during training, so evaluation needs no rescaling and stays deterministic. Training and evaluation differ in exactly one controlled way.

A model accidentally left in `train()` mode at evaluation gives noisy, degraded predictions — a classic silent bug. `model.eval()` before measuring anything.

Three controls that students often conflate

control	dropout	BatchNorm	autograd
model.train()	random masks	batch statistics; buffers update	unchanged
model.eval()	identity	stored statistics	unchanged
torch.inference_mode()	unchanged	unchanged	disabled

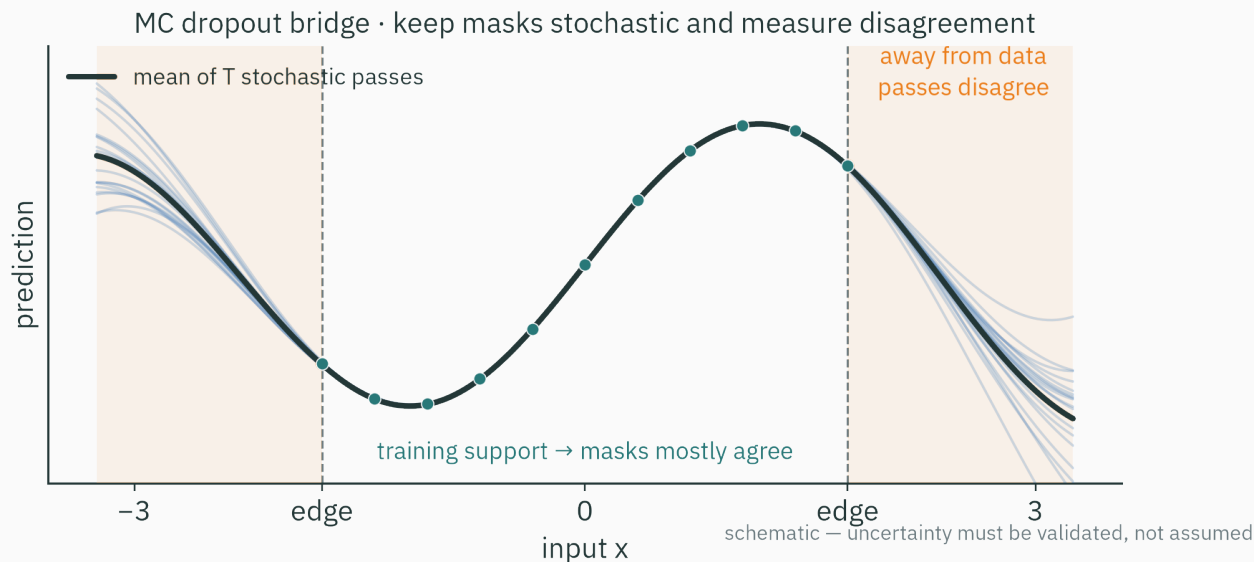
eval() changes module behaviour; inference_mode() changes gradient tracking. They solve different problems.

Where dropout earns its keep today

setting	current practice
wide fully-connected heads	classic sweet spot — where co-adaptation is worst
transformer blocks	small rates inside attention / FFN and on residuals
modern conv backbones	often secondary to strong augmentation and weight decay; classifier-head dropout remains common
whole residual branches	the same idea at branch scale: stochastic depth / DropPath

The knob is p : sweep it like λ and read **validation**. Too little may not help; too much destroys useful signal.

Standard inference collapses the masks; MC Dropout keeps sampling V VISUAL



Standard: dropout off → one deterministic prediction

MC: dropout on → T probabilities; report mean + spread

schematic: masks agree near training support and can disagree away from it · spread is a useful signal, not a guarantee of correctness

```
DROPOUT = (nn.Dropout, nn.Dropout1d, nn.Dropout2d, nn.Dropout3d)

def mc_predict(model, x, T=30):
    model.eval()  # BatchNorm uses stored statistics
    for module in model.modules():
        if isinstance(module, DROPOUT):
            module.train()  # only dropout stays stochastic

    with torch.inference_mode():
        probs = torch.stack([
            model(x).softmax(dim=-1) for _ in range(T)
        ])

    model.eval()  # restore deterministic evaluation
    return probs.mean(0), probs.std(0)
```

Do not call `model.train()` globally: BatchNorm would use the inference batch and update its buffers. Average **probabilities**, not logits. Functional dropout needs a separate explicit switch.

Ready now

why masks create different subnetworks; inverted scaling; train/eval modes; how to sample safely

Next lecture

predictive entropy; mutual information / BALD; Bayesian interpretation; calibration; failure cases

$$x \rightarrow \{m^1, \dots, m^T\} \rightarrow \{p^1, \dots, p^T\} \rightarrow \left(\underbrace{\frac{1}{T} \sum_t p^t}_{\text{mean}}, \underbrace{\text{spread}(p^{1:T})}_{\text{uncertainty signal}} \right)$$

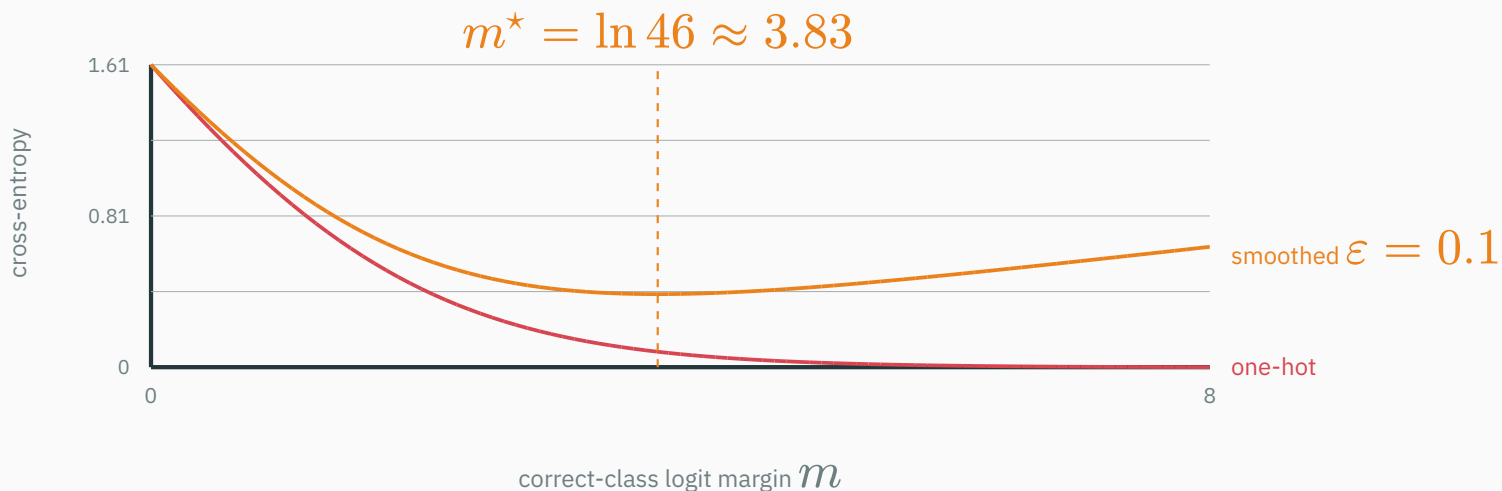
The mask was a training regularizer. Keeping it stochastic at inference turns it into a prediction distribution.

Label smoothing: soften the targets

One-hot targets demand infinite confidence



With target $y_b = 1$, cross-entropy wants $p_b = 1$ — which softmax only reaches as the correct logit races to $+\infty$:

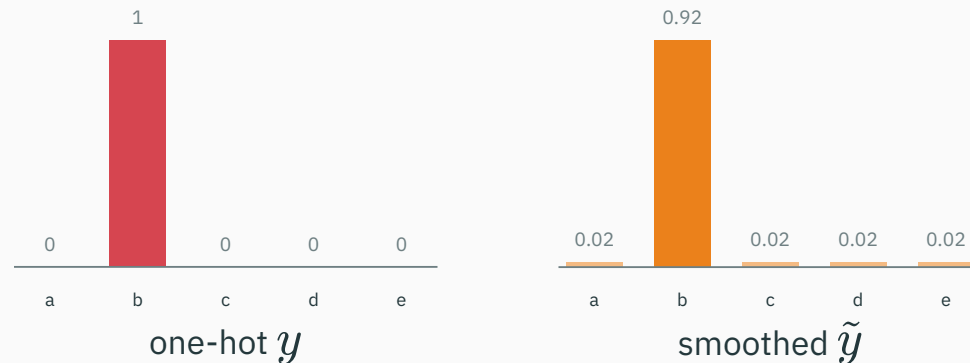


One-hot: the gradient never rests. **Smoothed:** a finite margin is **optimal**.

Move ε of the target mass off the correct class

$$\tilde{y} = (1 - \varepsilon) y + \frac{\varepsilon}{K} \mathbf{1}$$

For $K = 5$, $\varepsilon = 0.1$, correct class “b”:



Convention check: PyTorch’s `CrossEntropyLoss(label_smoothing=0.1)` mixes with the uniform distribution over **all** K classes, giving $0.92/0.02$ — not the $0.90/0.025$ of “redistribute to others”. Name the convention before comparing ε values. [PyTorch API reference](#)

```
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

model.train()
for images, class_ids in train_loader:
    logits = model(images)
    loss = criterion(logits, class_ids)
    optimizer.zero_grad(set_to_none=True)
    loss.backward(); optimizer.step()

model.eval()
with torch.inference_mode():
    val_logits = model(val_images)
    val_nll = F.cross_entropy(val_logits, val_class_ids)  # hard labels
```

Tune ε on validation NLL or the metric you actually care about — not on training loss.

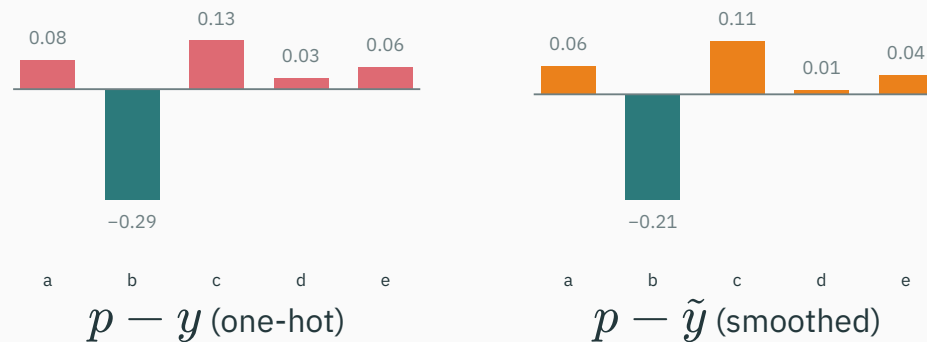
The smoothed training loss has a positive floor; do not compare its raw value with a one-hot run.

Where the softened optimum comes from

Zero gradient forces $p = \tilde{y}$, so at the optimum

$$\frac{e^{m^*}}{e^{m^*} + 4} = 0.92 \Rightarrow m^* = \ln 46 \approx 3.83$$

The mechanism, visible in the logit gradient $\nabla_z \ell = p - \tilde{y}$ at $z = (0.6, 2.8, 1.1, -0.4, 0.3)$:



negative bar = logit pushed up · smoothing stops pushing once wrong classes reach 0.02

Can help

tempers over-confident logits and can improve validation NLL, accuracy, or calibration — the effect is data- and recipe-dependent

Costs

output probabilities are pulled toward uniformity — softer interpretability, and distillation teachers can lose class-similarity structure (Müller et al., 2019)

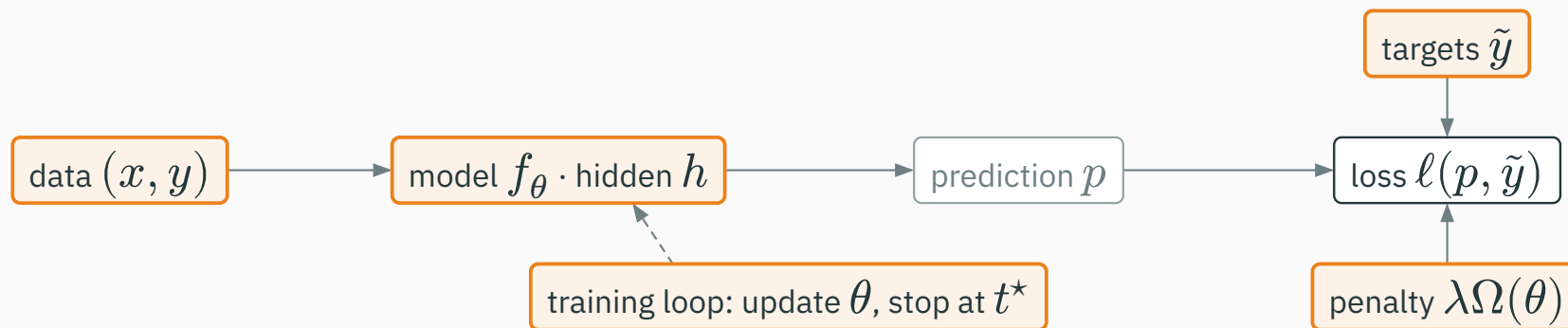
The knob is ε , the judge is **validation** — and for probability quality, measure calibration explicitly rather than admiring confidence. [Explore: interactive-articles/calibration](#)

Soft-target methods interact — do not stack them blindly

method	target source	what becomes softer
label smoothing	uniform distribution	confidence for every example
MixUp / CutMix	a second training label	confidence and behaviour between examples
both together	mixed target, then uniform mix	possibly too much target entropy

If MixUp or CutMix already produces soft targets, test label smoothing as a separate ablation. More regularization is not monotonically better.

One preference, many knobs



Different sites, one job: decide what the model does **where the data is silent — then let **validation** set the knob.**

The taxonomy, with each method's knob

site	methods	knob
objective	weight decay (L_2), L_1 , spectral penalties	λ
training loop	early stopping, minibatch noise	t^* , batch size
model / features	dropout, stochastic depth	p
data	crops · flips · jitter, Mixup, CutMix	strength, α
targets	label smoothing	ε
architecture	convolution, weight sharing, bottlenecks	design choice

Architecture adds no explicit penalty term, but it is not “free”: it restricts the function family and is still a validation-selected design choice.

Training regularizes even when you add nothing

implicit source	selection pressure it exerts
SGD minibatch noise	a stochastic trajectory with optimizer- and batch-size-dependent implicit bias
initialization + early phase	which basin training explores first (Lecture 6's init story)
architecture	some functions are easy to express, others practically unreachable
BatchNorm's batch statistics	per-batch jitter in activations — a side effect

Keep BatchNorm filed under **optimization** (Lecture 6): its job is trainability; mild regularization is a bonus. This is also why changing optimizer or batch size silently moves your **effective** regularization.

reproducible baseline → valid augmentation → weight decay → targeted extras

1	establish	log train/validation curves; restore the best checkpoint
2	encode	add domain-valid augmentation; inspect actual batches
3	control	sweep AdamW decay; compare at the same compute budget
4	only if needed	ablate dropout, MixUp/CutMix, or label smoothing one at a time

Methods reserved for specific problems:

stochastic depth / DropPath	dropout at residual-branch scale; standard in deep ViTs
spectral normalization	constrain layer operator norms (GANs, sensitivity control)
SAM	optimize the worst loss in a parameter neighbourhood
adversarial training	augment with worst-case perturbations — robustness, at a cost

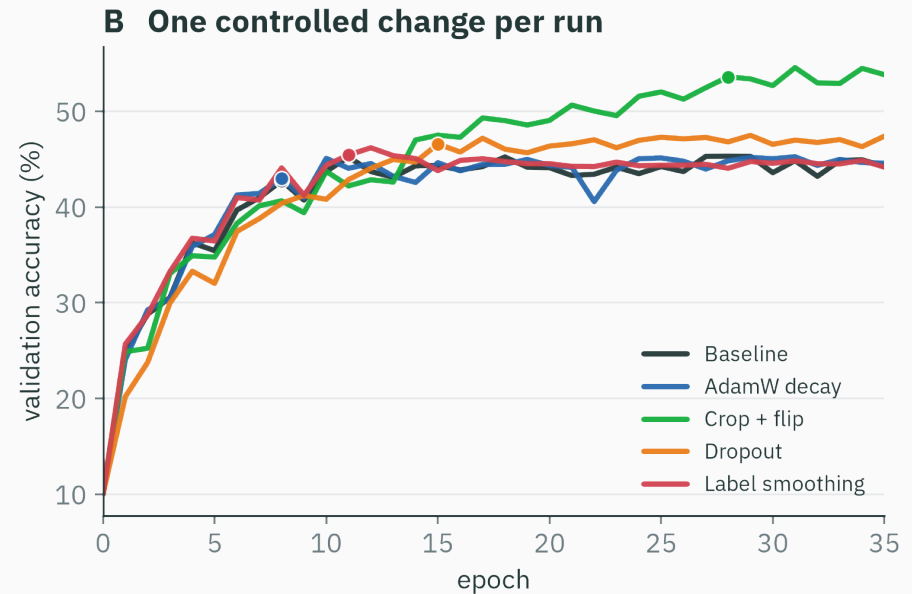
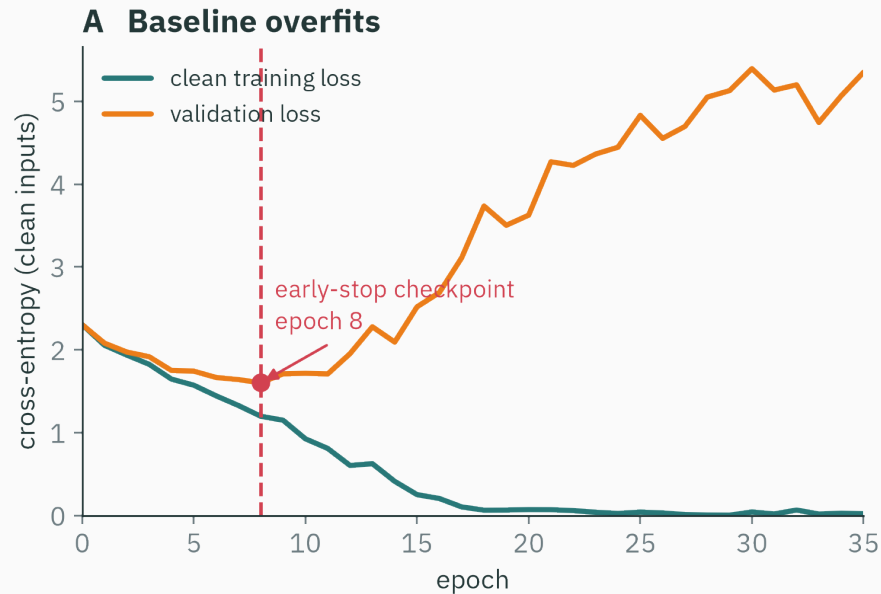
One change at a time, or the comparison identifies nothing. Sealed test set, evaluated once, after every knob is frozen.

What breaks: symptom → suspect → test

symptom	first suspect	controlled test
train and val both high	underfit / optimization	fix fit first (L5–L6); regularizing makes it worse
train ↓, val turns ↑	overfitting over time	restore best checkpoint; then sweep one knob
val better than train	train-only noise: dropout, aug, penalty in train loss	re-measure both in <code>eval()</code> mode, penalty excluded
great val, poor deployment	validation no longer represents reality	audit the split and the data pipeline, not the knobs
confident and wrong	target pressure	add smoothing; measure calibration

Regularization treats a diagnosed disease. It is not a vitamin you add by default in arbitrary doses.

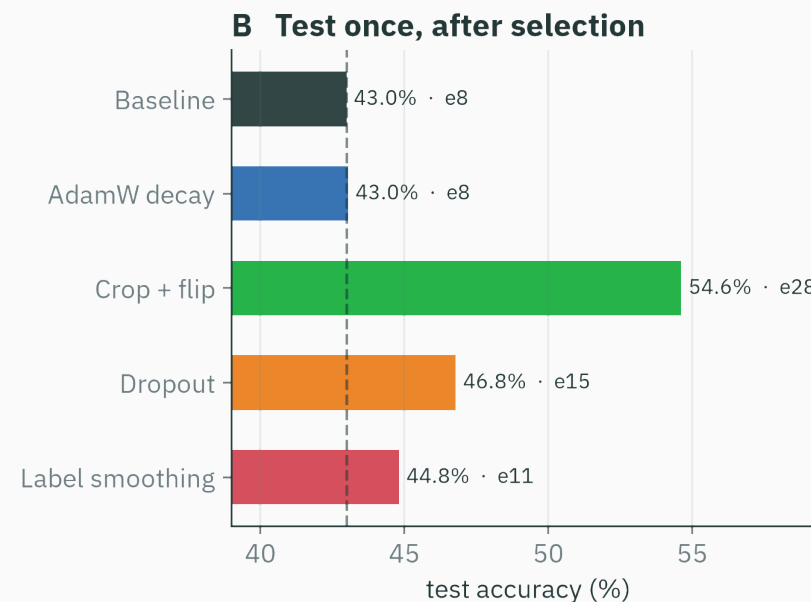
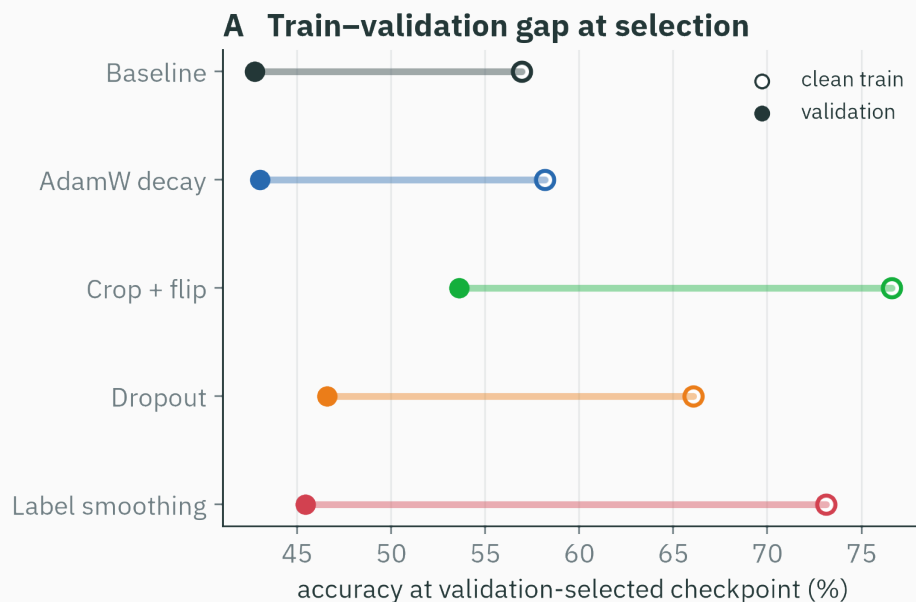
A controlled CIFAR-10 run: the baseline memorizes



single fixed seed · dots = checkpoint chosen by validation loss · test set not shown

2,000 stratified training examples · one fixed initialization and minibatch order · same 666,538-parameter CNN and 35-epoch budget · dots mark the checkpoint selected by validation loss.

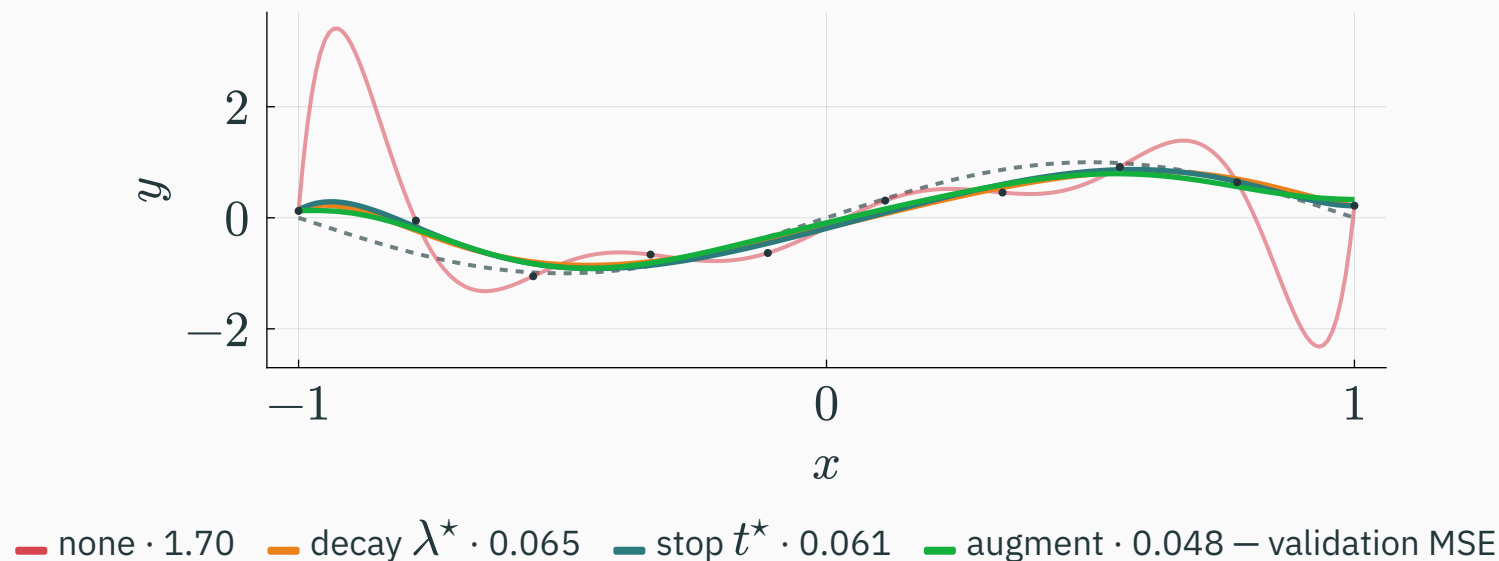
Same model, one controlled change per run



2,000 train · 5,000 validation · 10,000 test · one seed (descriptive, not a benchmark)

Crop + flip wins; dropout and smoothing help modestly; tested AdamW $\lambda_{\text{wd}} = 0.01$ is neutral. Suite and rules predeclared; each checkpoint frozen by validation before its reporting-only test. One seed $\neq$ benchmark.

The spine, cured three independent ways



Three different sites, three different knobs — the **same preference, and nearly the same cured model.**

Freeze every choice; only then open the sealed test set

model	choice made without test	val MSE	test MSE
interpolant	unregularized reference	1.704	1.187
weight decay	$\lambda^* = 0.001$ from validation	0.065	0.124
early stop	$t^* = 5000$ from validation	0.061	0.125
input jitter	policy fixed before test	0.048	0.113

The test column is a report, never a steering wheel. If it changes a knob, it has silently become another validation set.

SYNTHETIC · COMPUTED — seeded 10-point teaching case; every curve and number on this slide is recomputed from it at compile time

Exit ticket: five questions before you regularize

1. Which learning curves did you plot, and what do they diagnose?

3. What did validation choose — and did train loss get a vote?

5. Which implicit regularizers moved when you changed optimizer or batch size?

2. Which single site are you intervening on, and with which knob?

4. Is the test set still sealed, to be spent exactly once?

Reproduce: seed, splits, schedule, and every knob — logged.

If the answer to (1) is “none”, the prescription precedes the diagnosis.

Primary sources

- Zhang, Bengio, Hardt, Recht, Vinyals (2017), **Understanding deep learning requires rethinking generalization** — networks can fit random labels.
- Loshchilov & Hutter (2019), **Decoupled Weight Decay Regularization** — AdamW.
- Srivastava, Hinton, Krizhevsky, Sutskever, Salakhutdinov (2014), **Dropout**.
- Gal & Ghahramani (2016), **Dropout as a Bayesian Approximation** — the MC Dropout bridge for the next lecture.
- Zhang, Cissé, Dauphin, Lopez-Paz (2018), **mixup** · Yun et al. (2019), **CutMix** · Cubuk et al. (2020), **RandAugment**.
- Szegedy et al. (2016), label smoothing · Müller, Kornblith, Hinton (2019), **When Does Label Smoothing Help?**.
- Bishop (1995), **Training with noise is equivalent to Tikhonov regularization**; Belkin et al. (2019), double descent — the modern interpolation picture.

Seeded-spine figures are recomputed at compile time; the opening schematic, CIFAR panels, and controlled PyTorch ablation have documented provenance and scripts. See `lecture7/figures/PROVENANCE.md`; the companion Typst lab independently checks the regression numbers.

Regularization decides what the model does where the data is silent — you state the preference, validation sets the knob.

Next: keep dropout stochastic at inference — turn 30 subnetworks into a predictive mean and an uncertainty signal.

Next lecture · MC Dropout: uncertainty, calibration, and failure modes